

I. Mapa UWP

1. Autoryzacja

Wymagany jest klucz dla kontrolki mapy, wygenerowany na koncie developera www.bingmapsportal.com (usługa wygaszana, nieaktywna dla nowych użytkowników) albo w usłudze Azure Maps. Wygenerowany klucz służy do ustawienia wartości atrybutu *MapServiceToken* kontrolki *MapControl*. Jest także niezbędny do użycia usług mapy – wtedy należy przesyłać wartość klucza w zapytaniu.

Usługi Bing Maps dostępne są jako tzw. serwis REST – poprzez unikalne adresy URL. Bing Maps REST API umożliwia m.in.:

- Geokodowanie – zamiana adresu na współrzędne geograficzne i odwrotnie.
- Tworzenie tras – wyznaczanie tras dla samochodów, pieszych, ciężarówek i transportu publicznego.
- Mapy statyczne – generowanie obrazów map z pinezkami, trasami itp.
- Dane o ruchu drogowym – pobieranie informacji o incydentach drogowych.
- Sugestie lokalizacji (autosuggest) – podpowiedzi lokalizacji na podstawie wpisywanego tekstu.
- Strefy czasowe – pobieranie informacji o strefach czasowych dla danej lokalizacji.
- Dane wysokościowe – uzyskiwanie informacji o wysokości terenu.

Konta darmowe (Basic) mogą korzystać z usług do 30 czerwca 2025, płatne konta Enterprise – do 30 czerwca 2028. Microsoft zaleca migrację do Azure Maps, które oferuje podobne (i rozszerzone) funkcje.

Dla BingMaps usługa geokodowania dostępna jest pod adresem:

url = <http://dev.virtualearth.net/REST/v1/Locations?q={encodedAddress}&key={apiKey}>

Gdzie:

- apiKey = "TWOJ_KLUCZ_BING_MAPS";
- encodedAddress = Uri.EscapeDataString("ul. Gdańska 1, Bydgoszcz, Polska");

korzystając z obiektu *System.Net.Http.HttpClient* i metody *GetAsync*.

Wykorzystanie Azure Maps wymaga wysłania zapytania na adres:

url = \$"https://atlas.microsoft.com/search/address/json?api-version=1.0&subscription-key={subscriptionKey}&query={encodedAddress}";

2. Mapa, zoom, tryb wyświetlania. Menu

1. Rozpocznij tworzenie nowego projektu (Uniwersal Windows, Blank App)

2. Kontener *Grid*, trzy wiersze; wysokość *auto*, reszta, *auto*.

3. Wiersz 1: „NAWIGATOR”; strona: „mapa”

4. Wiersz 2: umieść kontrolkę *Map*, zmień jej nazwę na *mojaMapa*; bez marginesów, bez rozmiaru;

a. ustaw atrybut *MapServiceToken*

5. wiersz 3: wstaw *CommandBar* i zdefiniuj menu (przed „</Page>”: 3 x *AppBarButton*, separator, *AppBarButton*)

a. powiększenie skali:

Icon="Add" *Label*="powiększ"

Click="powMapa"

b. pomniejszenie:

Icon="Remove" *Label*="zmniejsz"

Click="pomMapa"

c. zmiana trybu mapy (widok mapy/satelity):

Label="satelita" *Click*="trybMapy"

podelement <*AppBarButton.Icon*>

z <*FontIcon Glyph*="S" *FontFamily*="auto" />

d. separator (*AppBarSeparator*)

e. button przejścia na stronę „Koordynaty”

dla każdego zdarzenia *Click* wciśnij F12 (generuje procedurę zdarzeniową)

6. Zapisz i uruchom aplikację.

7. Procedury zdarzeniowe

a. *powMapa*, *pomMapa* - powiększanie/pomniejszanie (zmiana skali mapy):

i. dla kontrolki mapy (jak się nazywa?) odpowiednio zwiększaj/zmniejszaj właściwość *ZoomLevel*,

b. dodaj przestrzeń nazw *Windows.UI.Xaml.Controls.Maps*

c. *trybMapy*: tryb wyświetlania mapy: modyfikuj dla mapy właściwość *Style*: z enum *MapStyle*: drogi (*Road*) / satelita (*AerialWithRoads*), procedura na przemian zmienia tryb oraz ikonę **S - M**

i. w procedurze utwórz obiekt o nazwie „ab” klasy *AppBarButton*

ii. przypisz do *ab* wartość parametru *sender* (rzutuj na *AppBarButton*)

iii. utwórz wskaźnik *fIcon* do *Icon* obiektu *ab*, rzutuj na *FontIcon*

iv. dla mapy - sprawdź, jaki aktualnie jest tryb – zamień na przeciwny:

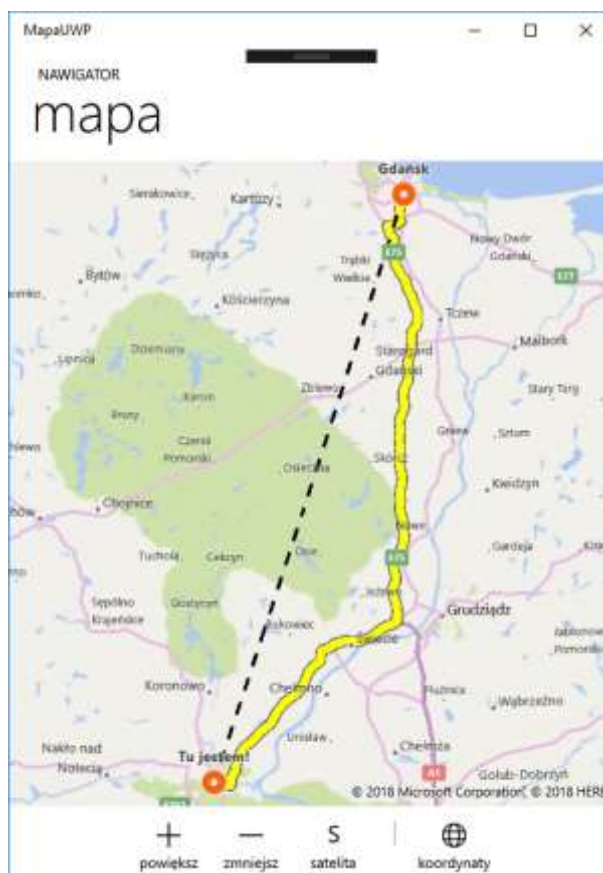
jeżeli: (*mojaMapa.Style* == *MapStyle.AerialWithRoads*) to zamień:

```
mojaMapa.Style = MapStyle.Road;
```

```
ab.Label = "satelita";
```

```
fIcon.Glyph = "S";
```

w przeciwnym przypadku przywróć tryb *AerialWithRoads*, napis *mapa* oraz ikonę *M*



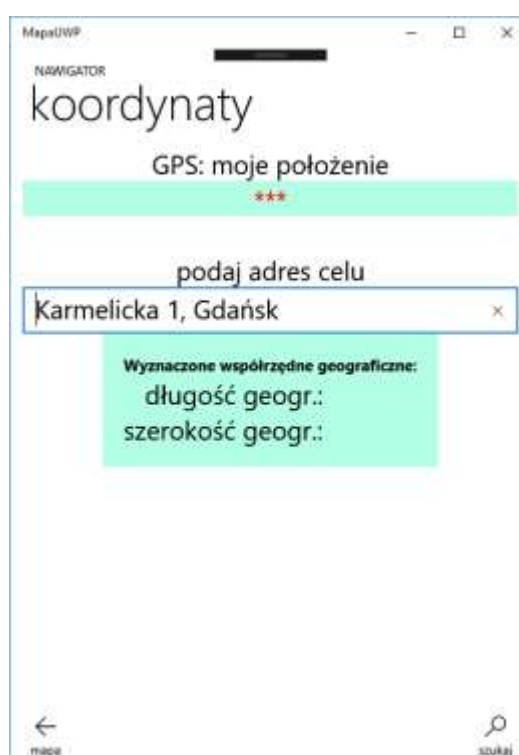
Geolokalizacja

Współrzędne punktu startowego(położenie telefonu) zostaną określone przez wbudowany GPS. Użytkownik poda adres punktu docelowego – usługa wyznaczy współrzędne geograficzne punktu docelowego.

Po powrocie do strony głównej oba punkty zostaną oznaczone na mapie przez znaczniki. Pomiędzy tymi punktami wytyczona zostanie trasa linią prostą oraz trasa przejazdu samochodem.

- 1) Na stronie MainPage – dla button-u przejścia na stronę „Koordynaty”:
 - a) ustaw etykietę na „koordynaty”,
 - b) dla zdarzenia *Click* napisz procedurę przejścia (nawigacji) do strony *Koordynaty* (metoda *Navigate* obiektu *Frame*)
- 2) Dodaj nową stronę o nazwie *Koordynaty* (Solution Explorer, nazwa projektu, menu podręczne, Add New Item – Blank Page), w której będzie:
 - a) układ 3 wierszy (jak w MainPage)
 - i) z nazwą aplikacji i strony
 - ii) drugi:
 - wyświetlenie współrzędnych bieżących z GPS-a (moje położenie) – etykieta *tbGPS*
 - wprowadzenie z ręki adresu docelowego (np. Karmelicka 1, Gdańsk) – pole tekstowe *txAdres*,
 - wyświetlenie wyznaczonych długości i szerokości geograficznej celu (etykiety *tbDlg* oraz *tbSzer*)
 - iii) przycisk powrotu (*Frame.GoBack()*) oraz przycisk wyznaczenia koordynat celu.

Bydgoszcz orient.: 17.978; 53.1245



- 3) Przygotuj klasę, która umożliwi przekazanie danych ze strony *Koordynaty.xaml* do strony *MainPage.xaml* (będą to współrzędne punktu startowego oraz docelowego) oraz adres celu
 - a) dodaj do projektu nową klasę (Add, Class) o nazwie *DaneGeograficzne*; usuń wszystkie instrukcje *using*, wstaw tylko:
`using Windows.Devices.Geolocation;`
 - b) w klasie *DaneGeograficzne* zdefiniuj:
 - i) dwa publiczne statyczne pola typu *BasicGeoposition* odpowiednio o nazwie *pktStartowy* oraz *pktDocelowy*
 - ii) jedno pole publiczne statyczne tekstowe „opisCelu”; ustaw na *null*
- 4) Zapisz wszystko (Ctrl+Shift+S), wróć do kodu strony *Koordynaty.xaml.cs*
 - a) dodaj przestrzeń nazw *Windows.Devices.Geolocation*
 - b) w metodzie asynchronicznej `void GdzieJaNaMapie()` odczytaj współrzędne aktualne z GPS-a:
 - i) zainicjuj obiekt mójGPS klasy *Geolocator* z właściwością *DesiredAccuracy = High*

- ii) odczytaj aktualne położenie – dla obiektu `mójGPS` wywołaj asynchroniczną (*await*) metodę `GetGeopositionAsync()`; wynik zapisz do obiektu `mojeZGPS` klasy `Geoposition`
 - iii) odczytaj długość i szerokość geograficzną punktu startowego
z `mojeZGPS.Coordinate.Point.Position` - odpowiednio `Latitude` i `Longitude`,
 - iv) odczytaną długość i szerokość geograficzną sformatuj i wyświetl w *tbGPS*,
 - v) zapisz do pola `pktStartowy` klasy `DaneGeograficzne`; zrób nowy obiekt klasy `BasicGeoposition`; zainicjuj w nim pola `Latitude` i `Longitude` odpowiednimi wartościami szerokości i długości geograficznej.
- 5) Wskaż w używanych zasobach urządzenia (w *application manifest*) użycie funkcjonalności lokalizacji – zaznacz `ID_CAP_LOCATION` w **WMAppManifest.xml-Capabilities**. Zapisz, skompiluj – jeżeli brak błędów - uruchom program.
- 6) Geolokalizacja podanego adresu (adres-> współ. geograf.) – usługa BingMaps:
- a) Button „szukaj celu”, Click – funkcja asynchroniczna,
 - b) Użyj usługi BingMaps (opis na pierwszej stronie) - w zapytaniu dodaj **"&output=xml"**
 - c) Dla zwróconej odpowiedzi (jaka to klasa?) – testuj `IsSuccessStatusCode`; jeżeli dobrze, to:
 - i) dla zwróconego wyniku zapytania, pole `Content` – wywołaj `Content.ReadAsStringAsync()`
 - ii) wynik parsuj do dokumentu xml: `XDocument.Parse(...)`
 - iii) utwórz obiekt klasy `XNamespace`
`XNamespace ns = "http://schemas.microsoft.com/search/local/ws/rest/v1";`
 - iv) użyj „ns” do odczytania wartości elementu „Name” (szukany adres):
`var adres= doc.Descendants(ns + "Name").FirstOrDefault().Value;`
 - adres zapisz w „DaneGeograficzne” jako pole „opisCelu”
 - analogicznie odczytaj element `Point` – wyciągnij długość i szerokość geograficzną, zrób z tego obiekt klasy `BasicGeoposition`, zapisz w „DaneGeograficzne” jako pole „pktDocelowy”
- 7) Zawartość klasy `DaneGeograficzne` testowo wyświetl na stronie

3. Mapa cz. II – znaczniki i trasa

- 1) Wróć do kodu strony głównej, Dodaj:
 - a) przestrzenie nazw
 - i) `Windows.UI.Xaml.Controls.Maps;`
 - ii) `Windows.Devices.Geolocation;`
 - iii) `Windows.Services.Maps;`
- 2) Przed wyświetleniem strony głównej - metoda:


```
protected override void OnNavigatedTo(NavigationEventArgs e)
```

 - a) sprawdź, czy są prawidłowe dane w `DaneGeograficzne` (jak?) – jeżeli nie, zakończ,
 - b) obiekt `pktStartowy` klasy `Geopoint` z „DaneGeograficzne.pktStartowy”
 - c) dla Bydgoszczy - znacznik `znacznikStart` klasy `MapIcon`
 - i) `Location = pktStartowy,`
 - ii) `Title = "Tu jestem!"`
 - d) analogicznie drugi znacznik w punkcie `DaneGeograficzne.pktDocelowy`
 - e) dodaj linię łączącą punkty `pktStartowy` oraz `pktDocelowy`
 - f) obiekt „`trasaLotem`” klasy `MapPolyline`
 - i) `StrokeColor = Windows.UI.Colors.Black,`
 - ii) `StrokeThickness = 3,`
 - iii) `StrokeDashed = true,`
 - iv) `Path` – kolekcja typu `GeoPath` - punkty wyznaczające początek i koniec:

```

Path = new Geopath(new List<BasicGeoposition> {
    DaneGeograficzne.pktStartowy,
    DaneGeograficzne.pktDocelowy
}),

```

g) dodaj do mapy znaczniki oraz „trasaLotem: dla kolekcji *MapElements* metodą *Add*

h) dla mapy ustaw centrowanie na *pktStartowy*, skala na 8:

```
await mojaMapa.TrySetViewAsync(new Geopoint(DaneGeograficzne.pktStartowy), 8);
```

i) co *await* oznacza dla metody, w której to piszemy?

- 3) Skompiluj, usuń błędy, uruchom, sprawdź (dla emulatora użyj lokalizacji – ustaw położenie aktualne na Bydgoszcz).
- 4) Odległość (*pktStartowy*, *pktDocelowy*):
 - a) dla *pktStartowy* użyj *GetDistanceTo*(*DaneGeograficzne.pktDocelowy*)
 - b) wyświetl komunikat z odległością w [km]
- 5) Uruchom, sprawdź działanie aplikacji.
- 6) Wyznaczanie trasy
 - a) Użyj biblioteki *BingMapsRESTToolkit*
- 7) Uruchom, sprawdź działanie aplikacji.

Zadania:

1. Użyj *try...catch(..)...* dla operacji wyznaczenia położenia punktu startowego oraz docelowego.
2. Dla strony głównej oraz Koordynaty użyj instrukcji zachowującej stan strony (tryb mapy, szukany adres). Ustaw wyświetlanie mapy tak, by cała trasa była widoczna (użyj [BoundingBox](#) z obiektu trasy).
3. Na stronie głównej wyświetl (użyj komunikatu) informacje o trasie: przewidywany łączny czas przejazdu, dystans oraz kilka początkowych manewrów.